

A comprehensive study on fault tolerance in stream processing systems

Xiaotong WANG¹, Chunxi ZHANF¹, Junhua FANG², Rong ZHANG¹, Weining QIAN¹, Aoying ZHOU¹

¹ School of Data Science and Engineering, East China Normal University, Shanghai 200062, China

² Advanced Data Analytics Laboratory of Soochow University, Suzhou 100029, China

Front. Comput. Sci., **Just Accepted Manuscript** • 10.1007/s11704-020-0248-x
<http://journal.hep.com.cn> on September 4, 2020

© Higher Education Press 2020

Just Accepted

This is a “Just Accepted” manuscript, which has been examined by the peer-review process and has been accepted for publication. A “Just Accepted” manuscript is published online shortly after its acceptance, which is prior to technical editing and formatting and author proofing. Higher Education Press (HEP) provides “Just Accepted” as an optional and free service which allows authors to make their results available to the research community as soon as possible after acceptance. After a manuscript has been technically edited and formatted, it will be removed from the “Just Accepted” Web site and published as an Online First article. Please note that technical editing may introduce minor changes to the manuscript text and/or graphics which may affect the content, and all legal disclaimers that apply to the journal pertain. In no event shall HEP be held responsible for errors or consequences arising from the use of any information contained in these “Just Accepted” manuscripts. To cite this manuscript please use its Digital Object Identifier (DOI(r)), which is identical for all formats of publication.”

A comprehensive study on fault tolerance in stream processing systems

Xiaotong WANG¹, Chunxi ZHANF¹, Junhua FANG², Rong ZHANG (✉)¹, Weining QIAN¹, Aoying ZHOU¹

¹ School of Data Science and Engineering, East China Normal University, Shanghai 200062, China

² Advanced Data Analytics Laboratory of Soochow University, Suzhou 100029, China

©Higher Education Press 2020

Abstract Stream processing has emerged as a useful technology for applications which require continuous and low latency computation on infinite streaming data. Since stream processing systems (SPSs) usually require distributed deployment on clusters of servers in face of large-scale of data, it is especially common to meet with failures of processing nodes or communication networks, but should be handled seriously considering service quality. A failed system may produce wrong results or become unavailable, resulting in a decline in user experience or even significant financial loss. Hence, a large amount of fault tolerance approaches have been proposed for SPSs. These approaches often have their own priorities on specific performance concerns, e.g., runtime overhead and recovery efficiency. Nevertheless, there is a lack of a systematic overview and classification of the state-of-the-art fault tolerance approaches in SPSs, which will become an obstacle for the development of SPSs. Therefore, we investigate the existing achievements and develop a taxonomy of the fault tolerance in SPSs. Furthermore, we propose an evaluation framework tailored for fault tolerance, demonstrate the experimental results on two representative open-source SPSs and exposit the possible disadvantages in current designs. Finally, we specify future research directions in this domain.

Keywords fault tolerance, performance evaluation, stream processing

1 Introduction

Real-time data processing is popularly deployed in various domains, including network intrusion detection, financial stock trading, military platoon tracking and so on. These applications produce data as continuous data streams. As the requirements of these applications make traditional database management system (DBMS) and batch processing system unsuitable, numerous stream processing systems (SPSs) have sprung up from both academic and industry. In the early 21st century, systems like Niagara [1], Aurora [2], STREAM [3] and TelegraphCQ [4] start the work in data stream management systems (DSMSs, one branch of SPS). Such DSMSs adopt a centralized architecture, and mainly serve sensor network applications as extensions of DBMSs [5]. Then the growth and intrinsic dispersity of streaming data stimulates the emergence of distributed stream processing systems (DSPSs) which provide advanced features such as fault tolerance and enhanced operator expressiveness. Representatives include Medusa [6] and Borealis [7]. Since the prevalence of cloud computing and HDFS/MapReduce [8] technique for massive-scale data, quite a few superior DSPSs have been developed to provide highly-scalable and highly-available services. Widely-used systems include Storm [9], Spark Streaming [10], Flink [11] and Samza [12]. Nowadays with 24/7 running requirements, SPSs usually require distributed and scalable deployment on clusters of commodity servers, such as Google Cloud [13], Amazon EC2 [14], Microsoft Azure [15] and so on. However, failures are ubiquitous as failure probability increases with the growing scale

of cluster and increasing running time [16]. For example, two independent studies [17, 18] respectively provide the estimations of failure likelihood for a cloud server to be 4% and 8%. In Google, it meets 1,000 individual node failures in the first year on each cluster; a 2,000-instance cluster encounters more than 10 node crashes per day [19]. Failures usually cause erroneous results or make system unavailable, resulting in a decline in user experience and even significant financial loss. For example, Amazon cloud have potentially suffered close to \$5 million in missed revenue for a single hour of offline time [20]. Therefore, fault tolerance is mandatory such that SPSs can recover rapidly and accurately enough to maximize system availability; meanwhile fault tolerance should be efficient enough to minimize the negative impact on system performance during failure-free runtime.

There have been a broad spectrum of approaches proposed to solve the problem of fault tolerance (FT) in SPSs. Most of the existing approaches focus on *fail-stop* failures. That is, if a failure happens on one server, it hangs up immediately and will be taken over by a health standby server. In general, fault tolerance in SPS is realized via replication, either the processing of data itself or the intermediate state of system. Accordingly, we group them into *active* scheme [21–25], *passive* scheme [26–34] and *hybrid* scheme [35–39]. Active fault tolerance in SPSs is inspired by the idea of state machine approach [40], which replicates the deterministic computation on multiple nodes. The critical step is to reach consensus among the replicas for the same input order. Paxos [41] algorithm is a well-known strong fault-tolerant technique for achieving consensus in distributed environment, but is too expensive in stream processing scenarios. Hence in stream processing, both primary and standby nodes actively run the same kind of service simultaneously. When the primary node is unavailable, one of standby nodes can instantly take over. Researchers propose different approaches for sorting the inputs/outputs of both primary and standby nodes in the same order. Passive fault tolerance is the most popular way in existing systems. In contrast to active scheme, only the primary node process the input streams and creates the checkpoints (a.k.a, snapshots) of essential recovery-related data to stable storage. Upon failure, a standby node reconstructs the state from the checkpoints and resumes the normal processing. Plenty of approaches have been explored for checkpointing, which differ in how to coordinate all the operators on the primary nodes to create checkpoints, when to issue a checkpointing request, what data to be included in a checkpoint and where to store the checkpoints. In recent years, some work have attempted to leverage the advantages of active and pas-

sive scheme, and proposed several hybrid approaches. These techniques differ in which operators are deployed with active/passive scheme and when to switch the scheme during the runtime.

This paper first present a comprehensive study on fault tolerance in stream processing systems. As shown in Table 1, some previous work has introduced the developments of this field, but is totally different from our work. Elnozahy et al. [42] provide a first survey on checkpointing-related approaches in message-passing systems. As message-passing systems differ from natural SPSs, this work can be considered as a supplementary of our article. Balazinska et al. [43] study fault tolerance issue for earlier generation of SPSs, e.g., Aurora [2] and Borealis [7], which are totally out-of-date. In recent years, Andre et al. [44], Heinze et al. [5] and Muhammad [45] provide an overview of fault tolerance in stream processing, but merely introduce fault tolerance implementations in several well-known systems. Thus, in this paper, we are the first to give a systematic and comparative in-depth analysis to representative fault tolerance techniques from both mature systems and literature research, e.g., the taxonomies, and the advantages and limitations of each technique. Furthermore, to systematically understand the quality of fault tolerance techniques, we first propose an evaluation framework to quantitatively measure their runtime overhead and recovery efficiency. It fills the gap that most of the existing work [46–53] only focuses on performance evaluation of SPSs during failure-free runtime. Finally, based on our thorough study and experimental comparisons, we conclude possible future research directions.

Table 1 Comparisons between our study and existing work

Survey	General Classification			Study Sample		Analysis	
	Active	Passive	Hybrid	Syst.	Lit.	Qual.	Quant.
Elnozahy et al. [42]		✓		✓	✓	✓	
Balazinska et al. [43]	✓	✓		✓		✓	
Andre et al. [44]		✓		✓		✓	
Heinze et al. [5]	✓	✓		✓		✓	
Muhammad et al. [45]	✓	✓		✓	✓	✓	
Our study	✓	✓	✓	✓	✓	✓	✓

The rest of this paper is organized as follows. Section 2 presents some background knowledge. We elaborate the design principles and implementations of each type of fault tolerance techniques in Section 3. In Section 4, we introduce our evaluation framework customized for fault tolerance and

conduct extensive experiments on representative DSPSs. Finally, conclusions and future directions are given in Section 5.

2 Basic concepts and definitions

In this section, we introduce some basic concepts of stream processing, and define the quality measurements for fault tolerance approaches.

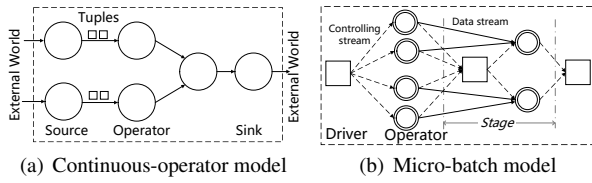


Fig. 1 Two types of stream processing models.

2.1 Stream processing model

SPSs continuously compute on data as it arrives. Data stream is unbounded sequence of tuples $\langle k, v \rangle$ where k is the key and v is the value. SPSs can be classified into two categories based on the processing model: (1) native *continuous-operator* model, such as Flink, Storm and Samza [12]; (2) *micro-batch* model, such as Spark Streaming, Drizzle [54] and Google Dataflow [55]. As shown in Fig. 1(a), data is separated from processing tasks in the continuous-operator model. User programs are converted into a directed acyclic graph (DAG, a.k.a., topology) of operators connected by data streams. Each operator encapsulates user-defined logics, and consumes(produces) tuples from input(into output) queues. An operator can be either stateless or stateful: a stateless operator (e.g., filter) consumes one tuple at a time and produces results barely based on it; a stateful operator (e.g., join) maintains intermediate results as internal states which are updated during processing subsequent tuples. Besides, there are two kinds of special operators in a topology, namely source and sink, which fetches(exports) tuples from(to) external world. Operators are usually parallelized into multiple instances and run on different servers for high throughput. On the contrary, as shown in Fig. 1(b), data is bundled with processing tasks in micro-batch model¹. Input streams are divided into series of discrete *t-seconds* micro-batches. User programs are converted into a DAG of *stages*. Within each stage, all the

parallel instances of an operator are scheduled by a centralized driver, consume the micro-batches and report the size of output results to the driver. Then the driver launches the next stage, and informs other operators about the data that they should fetch to process. Since SPSs with micro-batch model, e.g., Spark Streaming, are almost inherent fault-tolerant owing to the underlying resilient data structures and lineage information [10], in this paper we mainly focus on the analysis to SPSs with continuous-operator model, which are the real-time stream processing work.

Table 2 Quality measurements of fault tolerance

Runtime Overhead	Normal latency	
	Hardware resource	CPU
		Memory
Recovery Efficiency	Recovery accuracy	Network
		Recovery latency
		Precise
		Duplicate
		Lost
		Erroneous
		Disordered

2.2 Quality of fault tolerance

Researchers [46–53] have put much effort in benchmarking the characteristics (e.g., scalability) and performance (e.g., throughput and latency) of modern DSPSs. We investigate the systems under test, datasets, workloads and metrics of previous work. As we can see from Table 3, current stream benchmarks mainly focus on the evaluations during failure-free runtime, except that StreamBench [47] introduces a penalty factor for latency and throughput.

Although there is a rich body of research on fault tolerance of SPSs, there is no systematic work to evaluate and compare them in detail, which is urgent to consolidate the state-of-the-art research and to guide the development of efficient fault tolerance approaches. We address such a gap and first propose an evaluation framework to measure the quality of each fault tolerance approach. As summarized in Table 2, we define two types of new metrics, i.e., *runtime overhead* and *recovery efficiency*, to target evaluating fault tolerance. Obviously, extra overhead is inevitable during failure-free runtime, such as at least twice the hardware resource consumption in active approaches, or performance penalty, e.g., longer normal latency² incurred by checkpointing in passive approaches. Recovery efficiency can be measured from two aspects: on one hand, to ensure the high-availability of SPSs, it is better to keep recovery latency as low as possible; on

¹ Concentric circles in the same column are the parallel instances of an operator.

² To distinguish with recovery latency, we add a qualifier ‘normal’.

Table 3 Comparisons between our work and existing stream benchmarks

Benchmarks	System	Datasets	Workloads	Metrics
LinearRoad [46]	Aurora SREAM	Position reports Historical query requests	Toll processing Toll notifications Toll assessments Accident detection	Throughput
StreamBench [47]	Storm Flink Spark Streaming	AOL search data [56] CAIDA internet traces [57]	4 atomic operators Word count Distinct count statistics	Throughput Latency Penalty factor
Yahoo! StreamingBench [48]	Storm Flink Spark Streaming	Advertisements User clicks	Advertisement click analytics	Throughput Latency
YCSB Extension [49]	Storm Flink Spark Streaming	Advertisements User clicks	Advertisement click analytics	Throughput Latency
StreamBench [50]	Storm Flink Spark Streaming	Synthetic data	Word count Advertisement click analytics KMeans	Throughput Latency
RIOTBench [51]	Storm	Urban sensing data [58] Energy consumption [59] MHealth records [60]	ETL Statistics Model training Predictive Analytics	Peak rate Latency Resource usage Jitter
Benchmark Suite [52]	Storm Spark Streaming	Project Gutenberg [61] 1998 World Cup [62] Taxi traces [63]	Word count Click analytics Traffic monitoring	Throughput Latency Scalability Tuple loss Resource usage
Benchmarking DSDPS [53]	Storm Flink Spark Streaming	Purchasing records Advertisements	Windowed aggregation Windowed join	Throughput Sustainable throughput Latency
Our work	Storm Flink	English novel set Synthetic data	Word count π calculation Dummy topology	Throughput Normal latency Resource usage Recovery latency

the other hand, to produce correct results after recovery is also important, especially in the critical scenarios such as financial transaction. Hwang et al. [64] propose three recovery types: (1) precise recovery produces results identical to those without failure, which provide *exactly-once* semantics; (2) rollback recovery avoids the loss of results but allows duplication, which provides *at-least-once* semantics; (3) gap recovery is of weakest guarantee and allows the loss of results after failure, which provides *at-most-once* semantics. However, such a categorization isn't fine-grained enough to define the recovery accuracy. Thus on account of both *content* and *order*, we propose to integrate and redefine the recovery accuracy into five types, as enumerated in Table 2. Precise recovery perfectly satisfies these two points. For imprecise recovery may incur either lost/duplicated/erroneous contents of results, or disordered sequences of results. We give the quantitative measurements of metrics in Section 4.

3 Fault Tolerance Overview

In this section, we review three types of fault tolerance techniques in SPSs, make a detailed comparison among them and state the appropriate scenarios.

3.1 Active Scheme

Active fault tolerance is defined in Definition 1. To make sure that a standby node produces the correct results after it takes over the failed primary node, all the standby nodes need to keep consistent with the primary node before failures. Hence, the main challenge of active fault tolerance lies in replica coordination. Recovery procedure varies with different kinds of coordinations.

Definition 1 In active fault tolerance, each primary node is associated with $k \geq 1$ standby nodes which run the operators and consume the same input streams as the primary node, during failure-free runtime; once the primary node is

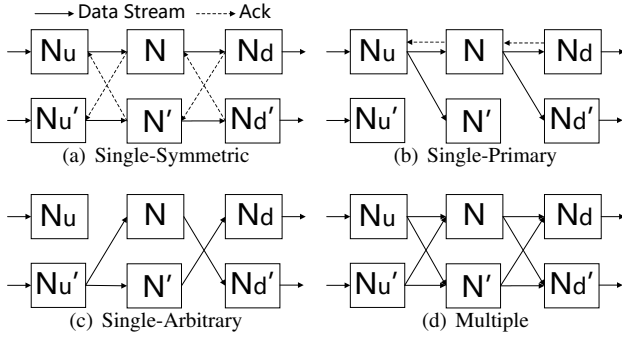


Fig. 2 Four typical kinds of replica coordination.

not available, one of the functioning standby nodes takes over virtually instantaneously.

3.1.1 Replica coordination

There are three key issues to be addressed in replica coordination. First, each replica of the same node should consume the *identical inputs*, which requires the same data set with the same order. Since nodes communicate with reliable and in-order network protocols (e.g., TCP), tuples never get disordered within one stream. Disorder happens on operators with more than one input streams (e.g., union, or operators that receive inputs from multiple partitioned upstream operators). Second, each replica should provide the *identical computation*. To this end, active approaches either assume that the operators are modeled as deterministic state machine [40], or tolerate certain inconsistency for nondeterministic operators [64], or merely consider the same set of output results [23]. Last, each replica should send the *identical outputs* downstream. As out-of-order upstream outputs can be sorted in downstream replicas, the identical outputs requires no additional process. For the sake of simplicity, we go through the case of $k = 1$, although all the active approaches can handle any k . As shown in Fig. 2, node N with its upstream neighbor N_u and downstream neighbor N_d constitutes one fragment of a topology. Each of them is associated with one standby node, i.e., N' , N_u' and N_d' .

Single-Source Scheme. In such a scheme, each downstream replica receives input from upstream either primary or standby replica during failure-free runtime. In terms of input subscription, we further group approaches into three types, as shown in Fig. 2(a)-2(c).

Single-Symmetric Scheme(SS). Each primary replica receives inputs from the upstream primary replica and sends outputs to the downstream primary replica. The standby

replicas take a symmetric way. Each replica buffers the results in the output queue before forwarding them downstream. When N receives a tuple r from N_u , it sends an acknowledgment ack_r to N_u' . Then, N_u' will remove the tuple with the same content of r from its output queue. As shown in Flux [21], it merges and sorts the inputs via the sequence number of each tuple, to ensure the identical order of inputs among replicas.

Single-Primary Scheme(SP). Both downstream primary and standby replicas receive inputs from the upstream primary. Standby replicas process the inputs, and buffer the results into output queues instead of forwarding them downstream. Due to storage space limitation, the challenge is to bound the size of output queues of standby replicas. Hwang et al. [64] first apply this idea in their work. In addition, they support non-deterministic operators, which means N and N' may generate different outputs (content and/or order). To truncate the output queues of each standby replica correctly, each replica must maintain the most recent tuple represented by the sequence number of each tuple (a.k.a, *watermark*), that contributes to the production of each output tuple. When N receives the acknowledgment for the latest tuple r processed downstream, it figures out the watermark w_r of r , and informs N' to trim all buffered output tuples that have an older watermark than w_r .

Single-Arbitrary Scheme(SA). Each replica receives the inputs from arbitrary upstream replica, as in Borealis [22]. As it merely supports deterministic operators, it is easier to bound the size of output queues than SP scheme, and requires no watermark mechanism. To achieve the identical inputs, periodical boundaries b_r which are special tuples containing only timestamps assigned by data sources and serve as punctuation tuples [65], are injected into stream. No tuples with smaller timestamps than b_r will appear after b_r in the same stream. Therefore, when a replica receives the boundaries from all the input streams, it sorts tuples whose timestamps are smaller than the oldest boundary.

Multi-source Scheme(MUL). Contrary to the single-source scheme, downstream replica is fed by multiple upstream replicas. Apart from sorting the inputs, each replica needs to filter duplicated tuples. Hwang et al. [23] propose a luxurious method in which each upstream replica broadcasts its output downstream. To identify the duplicated tuples, a specific filter is deployed at the entry of each node. The filter determines those tuples that later arrive from one stream replica as duplicates. Moreover, the filter is also implemented to provide identical inputs. It adopts the similar sorting mechanism as Borealis [22]. However, it sorts disordered streams

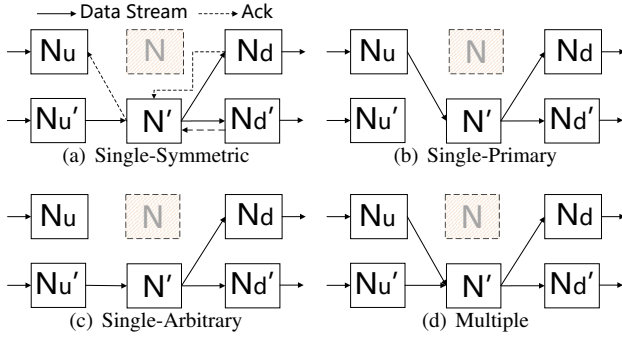


Fig. 3 Recovery procedure with different replica coordinations.

only when necessary, because it mainly focuses on the order-insensitive operators.

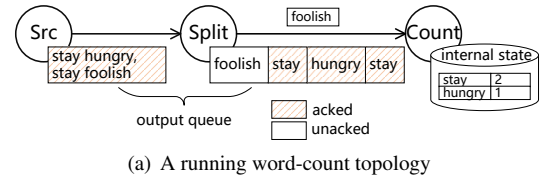
3.1.2 Recovery procedure

Once the primary replica N fails or the link connected to it is broken, it launches recovery processing by these four active schemes as illustrated in Fig. 3. For SS scheme, as shown in Fig. 3(a), N_d reverses its input connection to N' , while N_u continues processing its own inputs and acknowledgments from N' (for truncating the *acked* tuples in output queue), without forwarding any tuples downstream. Both N_d and N'_d will send acknowledgments to N' for tuples received. For SP scheme, as shown in Fig. 3(b), N' takes charge of forwarding output streams to both downstream replicas and asks the upstream primary to replay tuples in output queues that have not been acknowledged by the failed primary. For SA scheme, as shown in Fig. 3(c), if the downstream nodes already have other inputs (e.g., N_d and N'_d are originally fed by N'), nothing happens. Otherwise, the connected downstream nodes of the failed primary node need to find a functioning standby node of N . For MUL scheme, as shown in Fig. 3(d), it requires no recovery as the remaining replicas continues processing as if no failure happens.

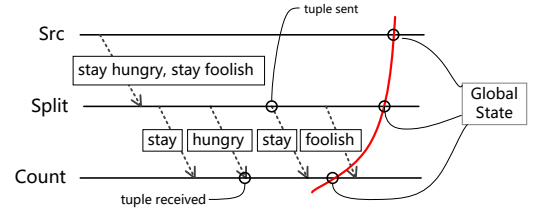
3.2 Passive Scheme

In contrast to active scheme, standby nodes in passive scheme do not process inputs during failure-free runtime. Passive fault tolerance is defined in Definition 2. Intuitively, checkpointing procedure is responsible for the largest part of performance overhead. Recovery procedure varies with different deployments of standby nodes.

Definition 2 In passive fault tolerance, operators on the primary node create checkpoints of internal state and/or other necessary information and store them to the stable storage



(a) A running word-count topology



(b) Consistent cut of global state

Fig. 4 An example of consistent global state: shaded bars in the output queue refer to the output tuples that have been received and acknowledged by downstream operators and should be pruned from the output queue; white bars refer to in-flight output tuples; the internal state maintains the intermediate results of stateful operator.

during failure-free runtime; once the primary node fails or becomes unavailable, a standby node initiates the corresponding operators, reconstructs the state from checkpoints and resumes the normal processing in recovery procedure.

3.2.1 Checkpointing procedure

The main prerequisite of checkpointing is to capture and copy the *global state* of system to stable storage, which is the collection of all the operators' internal states and in-flight tuples on communication channels. A *consistent* global state is the one that if the state of an operator involves the receipt of a tuple, then the state of the corresponding sender operator must involve the sending of that tuple [66]. Fig. 4 gives an example of consistent global state of a word-count topology. *Split* operator splits the sentence received from the *Source* operator and emits each word to the *Count* operator. *Count* has received the first three words, updated its internal state and acknowledged *Split* to remove these words from the output queue. As the tuple "foolish" is still in-transit, *Split* will buffer it in the output queue. Fig. 4(b) presents the corresponding consistent global state in the form of the red curve line called *cuts*. Each straight line represents the timeline of an operator, and the dotted arrows denote tuples sent and received between operators. Hence, A cut can be viewed as a red line that crosses the timelines of all the operator. The content of a consistent global state is the current internal state of each operator at the point in its timeline that the cut crosses, i.e., the count table as well as the in-flight "foolish".

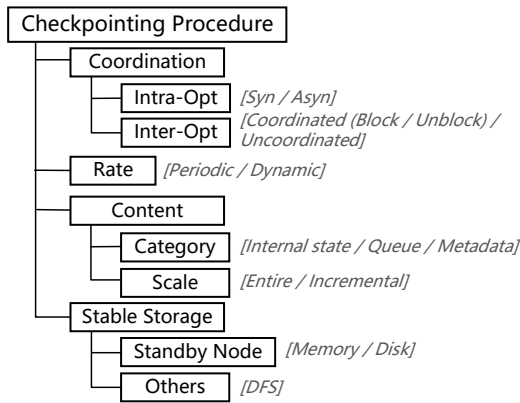


Fig. 5 Four building blocks of a checkpointing procedure.

A consistent global state created during failure-free runtime ensures the precise recovery, but requires the strict coordination among operators. Thus, some checkpointing techniques prefer the availability and create checkpoints without consistency guarantee. We abstract four key aspects shown in Fig. 5 that distinguish different checkpointing techniques: (1) checkpointing coordination decides how the operators interwork with each other; (2) checkpointing rate decides when to issue a checkpointing procedure; (3) checkpointing content decides what data to be involved in a checkpoint; (4) stable storage decides where to save checkpoints.

Checkpointing Coordination. We discuss the checkpointing coordination from two perspectives: intra-opt coordination refers to the interwork between the normal processing and checkpointing within a single operator; inter-opt coordination denotes the interaction among different operators.

Intra-Opt. When an operator begins to create a checkpoint, it suspends its normal processing until the checkpoint is saved to stable storage. Hence, checkpointing is *synchronous* with normal processing. Obviously, it leads to longer latency. To address the problem, Kwon et al. [27] adopt the idea of copy-on-write³⁾ to make checkpointing *asynchronously* with normal processing. When checkpointing begins, the operator makes a copy of its state and resumes the normal processing at once. The subsequent state modifications act on the copy while saving checkpoints to stable storage is performed in background. Asynchronous checkpointing reduces the extra normal latency, but consumes more memory.

Inter-Opt. Uncoordinated checkpointing has no interaction among the operators and allows each operator to decide when to make checkpoints independently. Wang et al. [67] propose to create a checkpoint when the amount of operator

state is small, which is also applied in Yahoo! S4 [68]. The main advantage of uncoordinated checkpointing is its simplicity and low runtime overload. But there are three serious drawbacks: (1) it is possible to incur domino effect [69], which may lead to lost a large quantity of work; (2) each operator is forced to maintain multiple checkpoints, and only one of them is involved in recovery procedure; (3) the recovery procedure is more complicated in that it needs to choose an appropriate checkpoint from each operator to construct a consistent global state of system. To address these problems, coordinated checkpointing is proposed. Each operator merely maintains one checkpoint on the stable storage. There are two kinds of operator teamwork:

- Blocking the normal processing of the entire system until checkpointing is done is a straightforward way. Source operators stop fetching new tuples from external sources. After that, each operator drains up the communication channels and accomplishes the checkpointing operation. Examples include Naiad [70].
- Inspired by the Chandy-Lamport distributed snapshot algorithm [66], special tuples (e.g., *marker* in Chandy-Lamport algorithm [66], *virtual line* in [64], *barrier* in Apache Flink [11]) with version identifiers are injected into data stream at source and flow with normal tuples. When an operator receives such special tuples from all of its input streams, it creates a checkpoint and then emits a special tuple with the same version identifier into all of its output streams. Downstream operators perform the same operation in the same way. In addition, Yu et.al. [28] propose another special tuple which is in the form of acknowledgments sent from downstream operators. The sink operator issues checkpointing first, and propagates acknowledgments upstream to inform other operators removing the corresponding tuples buffered in their output queues that have been successfully received and processed, and their states have been reflects into checkpoints at sink operators.

Block coordination is easy to implement, but evidently violates the property of continuous uninterrupted processing of SPSS; while unblock coordination improves system availability and is more preferable in production, but complicates the implementation.

Checkpointing Rate. In general, the rate of issuing a checkpoint request can either be periodic or dynamic.

Periodic. Periodic checkpointing is the most common way which specifies a global constant interval between two consecutive checkpointing requests. Determining the checkpoint

³⁾ <https://en.wikipedia.org/wiki/Copy-on-write>

interval has great impact on the performance. Smaller interval reduces the size of checkpoints (i.e., the size of operator's state and/or other necessary data) and shortens the recovery latency in turn, but incurs frequent interference with normal processing; while larger interval has less impact on normal processing, but lengthens the recovery latency accordingly due to larger size of checkpoints.

Dynamic. Dynamic checkpointing requires no constant checkpointing interval, but relies on some customized trigger mechanism. Huang et al. [32] propose that each operator triggers the checkpointing once its state divergence exceeds the user-defined threshold. Each operator maintains a copy of the latest checkpoint in memory. When an input tuple updates the current state, it computes the divergence between the current state and the copied one. The state checkpointing will be deferred until the divergence exceeds the user-defined threshold θ . Only limited built-in data structures are provided to calculate the state divergence, such as the difference of two values for numeric variables, Manhattan distance for vector, and difference of the numbers of keys in data set. Millwheel [71] takes an exhaustive (default) way to trigger the checkpointing. Each operator issues a checkpoint for every single state change during processing.

Checkpointing Content. We explore the content from two perspectives: data type and data scale.

Data Type. There are three types of data being included in a checkpoint: (1) Internal states are variables in memory that are updated as the operator processes tuples. It is also known as the intermediate results of the operator, such as the count table in Fig. 4(a) for *Count* operator counting the number of received tuples. Note that the internal state isn't the entire memory image of the operator which can be huge. (2) Input queue buffers the tuples that have not been processed by operators. Output queue buffers tuples that are in-transit or have been received by downstream operators but have not been acknowledged. In [64], both input and output queues are checkpointed. More specifically, TimeStream [72] backups two kinds of data in a checkpoint for each operator, namely state dependency (i.e., the collection of input tuples that makes the operator reach a specific state) and output dependency, i.e., the collection of input tuples that makes the operator produce a specific output starts from a specific state. In [32], only input queue is involved. Most of the passive fault tolerance approaches exclude input queues from checkpoint to significantly reduce the size of checkpoint. (3) Metadata is the abstraction of other recovery-related information. For example, in uncoordinated checkpointing, in order to rollback the system to a global consistency state, Bhargava et al. [73] sug-

gest the operator needs to record the dependencies among its checkpoints. In Flink [11], the offset of input tuples whose effects have been saved should also be included into the corresponding version of checkpoint.

Data Scale. Encapsulating the *entire* internal state into a single checkpoint is the simplest way, but consumes more storage and communication bandwidth due to rewriting portions of the states incurs minor changes between two consecutive checkpoints. To improve the performance, *incremental checkpointing* [74] is adopted to mitigate the disruption in normal processing. In summary, entire checkpointing is easy to implement but consumes resource, especially when the state is with gigabytes or even terabytes but has minor changes between two checkpoints. Incremental checkpointing decreases the size of backup but complicates the recovery processing due to the state integration.

Stable Storage. Saving checkpoints has two common choices which are to use standby nodes or distributed file systems.

- During failure-free runtime, the checkpoints of operators are saved in either *memory* or *disk* of standby nodes. Keeping the checkpoints in memory rather than in disk has the advantage of reducing recovery latency by skipping loading and de-serializing checkpoints from disk. But since SPSs mostly process streams in memory, memory is essentially a critical resource.
- Examples include Google File System (GFS) [75] and Hadoop Distributed File System (HDFS) [76], not only overcome the performance challenge of typical disk access, but also optimize the read and write of large data volumes. Moreover, they are inherently fault-tolerant.

Compared to DFS, using standby nodes for saving checkpoints can reduce the network bandwidth in that it directly retrieves checkpoints locally, but cannot survive the failures of standby nodes at the same time. Hence, DFS are more robust and reliable.

3.2.2 Recovery procedure

Recovery in passive fault tolerance is a *rollback* process that aims to restore the system execution back to some consistent point (intermediate or exactly pre-failure) captured during failure-free runtime. We break down the recovery procedure into two phases, namely deploying the standby nodes and reconstructing the operator states.

Standby Deployment. We go through an example to discuss how to deploy standby nodes when failure happens. As

shown in Fig. 6, we assume each node runs only one operator. Node N' serves as the standby of N . N consumes the input stream from its upstream N_u and forwards the output stream to its downstream N_d . To make N' correctly resume the normal processing when N fails, there are three steps that must be taken: (1) *operator initiation*, N' should restart the operators on the failed N ; (2) *connection establishment*, the original upstream/downstream node N_u/N_d should switch the connection to N' ; (3) *checkpoint retrieval*, N' should load the checkpoints from the stable storage to rebuild the state of N . Based on when these steps are taken, we specify four kinds of standby deployments inspired by [39].

Hot Deployment. The replica of the operator on the primary node N is deployed on standby node N' in advance. N' keeps running even its utilization is low most of the time. It still receives the same input tuples but just puts them into the input queue without any processing. The checkpoint of the operator on N is stored in the memory of N' to avoid the expensive disk I/O operations for storing/retrieving checkpoints, and corresponding outdated tuples in the input queue should be pruned. When N fails, N' should first establish the connection to the original downstream node N_d . As the checkpoint is kept in memory of standby node, it omits the steps of retrieving checkpoint from disk or third party and de-serializing the data structures. Finally, tuples in the input queue are replayed and then new incoming tuples are consumed.

Cold Deployment. Compared to hot deployment, the only difference is that no tuples are sent to the standby node N' . Accordingly, when N fails, the connections from upstream node N_u as well as downstream node N_d should be established, and then tuples buffered in the output queue of N_u are replayed.

Deployed Deployment. Likewise, the replica of the operator on N is already deployed on N' . However, checkpoints of N are not stored in the memory of N' , but in the disk or distributed file system. When N fails, it needs to establish the connections between N' and N_u/N_d as well. Then the checkpoints are loaded and de-serialized into the memory, and tuples in the output queue of N_u are replayed.

Vacant Assignment. The standby node N' is deployed on demand after a failure. In other words, it does not pre-deploy the binary code to execute the operator on N . Moreover, the checkpoints of N are not stored in the memory of N' as well.

Table 4 presents the differences of each standby deployment. The tick denotes the step is required. In summary, hot and cold deployments shorten the recovery latency owing to the unnecessary of operator initialization and checkpoint

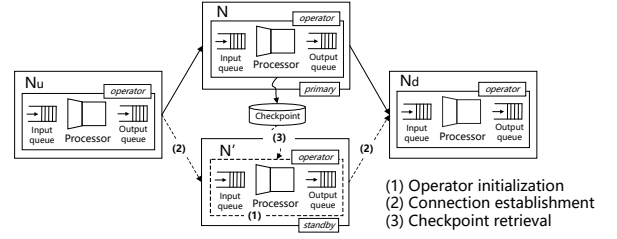


Fig. 6 The step-by-step diagram of standby deployment.

retrievement, especially hot deployment further reduces the latency as it requires no connection establishment and processes the replayed tuples directly in situ without network transmission. However, if the accumulated states are significantly large, memory consumption becomes a problem. The remaining two types of standby deployments address this problem but incur larger recovery latency due to checkpoint retrieval and de-serialization. Deployed deployment outperforms vacant deployment as the latter needs to deploy the instance replica first.

Table 4 Differences among four standby deployments

Deployments \ Steps	Operator Initiation	Connection Establishment	Checkpoint Retrieval
Hot			
Cold		✓	
Deployed		✓	✓
Vacant	✓	✓	✓

State Reconstruction. The state here is not just limited to the internal state of operators, but the processing progress of the system before failure. After the standby node has finished the deployment of operator replicas and the establishment of network link, it needs to invoke state reconstruction. The general pattern of state reconstruction consists of three steps: (1) retrieving the appropriate checkpoint from stable storage, (2) de-serializing it and (3) replaying tuples, if necessary. But in some cases, not all steps are required. For example, when the checkpoints are stored in the memory of standby node, the checkpoint retrieval and de-serialization are not involved. SGuard [27] skips the de-serialization step owing to its memory management middleware which provides special data structures. The tuple replaying differs significantly by the influence of the checkpoint content. For hot standby assignment or those which includes input queues in their checkpoints, the take-over standby simply needs to reprocess the tuples in input queues locally or from the de-serialized checkpoint. For those which include output queue in checkpoints, the upstream neighbors of the failed instance will replay its output queue to the standby node. For those which

only checkpoints the state of operator, it depends on the external fault-tolerant message queue, e.g., Apache Kafka, to replay tuples which were ingested from the point of the latest checkpoint to the point of failure. What's more, to achieve complete or eventual consistency, it requires a filtering component for each operator to eliminate the duplicated tuple it has met before failure.

The scope of rollback can either be *partial* or *global*. Partial one only involves the failed operators without affecting the other healthy ones. The standby instance retrieves and de-serializes the latest checkpoint that the primary created, and replays the buffered tuples (from input/output queue depending on the standby assignment). On the contrary, global rollback forces all the operators to reset back to a checkpoint. Even if only one operator fails, all the other healthy ones as well as the take-over standby node coordinate to rollback to their latest checkpoints together to form a global consistent state of system. The representative implementations are given [73] and Flink [11] respectively. In [73], each operator maintains multiple checkpoints. When an operator fails, all the operators will provide one checkpoint to form the global consistent snapshot. Rollback-dependency graph [73] and checkpoint graph [67] are two well-known techniques to calculating the global consistent snapshot based on the dependency information piggybacked in the checkpoint as mentioned above. In Flink, because a global consistent snapshot is already created during failure-free runtime, hence when an operator fails, all the operators (including the take-over standby) can just rollback to their latest checkpoint with the same version.

3.3 Hybrid scheme

For active fault tolerance, instant recovery comes at the expense of costly resource consumption and complicated implementation, while passive fault tolerance sacrifices longer recovery latency to save considerable resource. Intuitively, there has also been recent effort in the area to play the greatest strengths of both active and passive fault tolerance [35–39], for which finding an optimal assignment strategy is the key issue.

StreamMine3G [39] combines six fault tolerance approaches in a single system, two for active and four for passive. To adaptively switch the fault tolerance approach during runtime, it adopts a simple estimation model which accumulates the time of each decomposed recovery step. Users are required to specify a recovery latency threshold, the type of recovery accuracy, and optionally a cost weight vector

for resource consumption. StreamMine3G first starts with one active fault tolerance approach, and then periodically re-evaluates the performance metrics to choose an optimal approach. More specifically, the re-evaluation involves four steps: (1) computing the recovery latency for each approach, (2) filtering out those approaches whose expected recovery latency exceeds the threshold, (3) ranking the remaining ones by the expected resource consumption, and finally (4) choosing the most efficient one and triggering a switch, if necessary.

Heinze et al. [36] leverage machine learning techniques in an estimation model to evaluate the expected recovery latency of each operator. It first periodically samples the historical state size, normal latency and output queue length of each operator. Then all the samples are clustered into k -sized clusters, each of which is summarized through a centroid and updated online by newly captured normal latency. To estimate the expected recovery latency, it first identifies the closest cluster via the distance between the centroid and current operator, then adopts a linear regression or averages all the sampled recovery latency to figure out the final estimation value. If the expected value is above(below) upper(lower) threshold, current operator will switch to active(passive) fault tolerance to decrease future recovery latency(runtime overhead).

Li et al. [38] propose a PPA (passive and partially active) approach. All the operators adopt passive fault tolerance, but a selected set of operators also adopt active fault tolerance. When the failed operators contain those without active replicas, tentative results will be produced but can be rectified using techniques proposed in [22] after finishing the passive recovery process. To figure out which operators should be actively replicated, it provides a dynamic programming algorithms and two heuristic algorithms. All the solutions strive to maximize the quality of tentative results based on an output loss model.

3.4 Discussion

For active fault tolerance, all schemes require at least k times the hardware resources, and especially in *MUL* scheme which consumes k^2 times the network bandwidth. *SS* and *SP* scheme can provide precise recovery accuracy but incur higher recovery latency, because they strictly favor recovery accuracy by requiring at least one fully copy of topology to continue processing. The other two schemes prefer lower normal latency and recovery latency: *SA* scheme may generate imprecise results tentatively, and then correct them for convergent opera-

tors; *MUL* scheme only provides duplicate or disordered results.

For passive fault tolerance, it significantly reduce the hardware resources owing to the checkpointing mechanism. Meanwhile, reconstructing state from checkpoints worsens the recovery latency. Although scheduling checkpointing with normal processing consumes more latency, it is somewhat simpler to implement checkpointing mechanism than active fault tolerance. Quality of passive fault tolerance approaches varies with the four building blocks of checkpointing. For example, passive approaches with coordinated checkpointing are more preferable for key applications (e.g., banking business) that require precise results; while statistics applications (e.g., hot topics analysis) are inclined to uncoordinated checkpointing to gain instant response. Applications with large window computations (e.g., 2-hour-window) prefer incremental checkpointing to reduce the overhead when preparing and transferring checkpoints.

For hybrid fault tolerance, although the original intention is splendid, it is nontrivial to prove the efficiency and accuracy during scheme switch and scheme assignment.

In general, expensive active fault tolerance is inapplicable in the context of large-scale production deployment; economical passive fault tolerance is more common nowadays, but still suffers from the shortcomings of checkpointing; hybrid fault tolerance is being explored to enhance the cost effectiveness.

4 Evaluation framework and results

In this section, we first introduce our evaluation framework, including four quantitative measurements of both general and fault-tolerance-specific metrics, five workload suites with a set of configurable knobs. Although active fault tolerance provides excellent recovery, it consumes tremendous resource, which is not applicable for massive stream processing. Moreover, the vast majority of applications nowadays are latency tolerant. Thus current SPSs abandon active fault tolerance alone, and we focus on the evaluation of passive fault tolerance in this paper. As we introduce in Section 1, the underlying data structure with its linear graph makes SPSs with the micro-batch model inherently fault-tolerant. In other words, SPSs such as Spark Streaming provide no *operator-level* fault tolerance. Therefore, we apply the evaluation framework on two well-known SPSs with continuous-operator model, i.e., Flink and Storm. Our experiments aim to investigate the design principles of fault tolerance mech-

anism in each system, and to compare their fault tolerance performance in terms of runtime overhead and recovery efficiency.

Fig. 7 shows the overview of our evaluation framework. In-memory *Data Generator (DG)* injects tuple on-the-fly into *Data Broker (DB)* and controls the input rate as well as skew distribution. SPSs under test is isolated from *DG* and *DB*. It executes workloads with configurable knobs, fetches input streams from *DB* and exports processing results to *DB* as well. *Metrics Collector* is integrated into *DB* to collect the measurements of metrics.

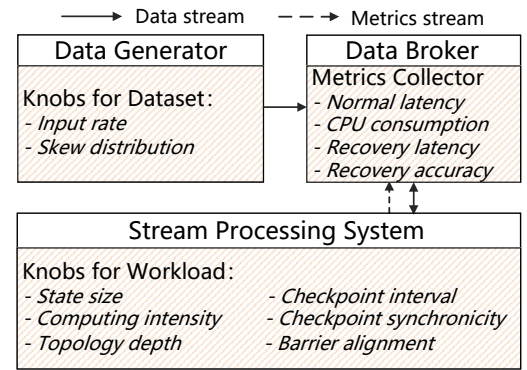


Fig. 7 Overview of evaluation framework.

4.1 Metric measurement

We identify and formalize the quantitative measurements for both general and fault-tolerance-specific metrics.

Normal Latency (NL). As defined in Equation 1, *NL* is the average duration (L_r) from the moment when each tuple r is generated at *DG* (T_r^{DG}) to the moment when all its causally dependent tuples generated at the sink operators (T_r^{Sink}). T_r^{DG} is known as event time [11], and thus *NL* is also called as event-time latency.

$$L_t = T_t^{DG} - T_t^{Sink}$$

$$NL = \overline{\sum L_t}. \quad (1)$$

Resource Consumption (CPU). Resource overhead often results from three aspects: scheduling checkpointing procedure and preparing operator state, which consume CPU cycles; maintaining operator state, which consumes memory; transmitting checkpoints to stable storage, which consumes network bandwidth. During experiments we observe that the consumption of memory and network bandwidth is proportional to the size of checkpoints. Therefore, we mainly focus on CPU. We monitor CPU every second by Ganglia⁴,

⁴ <http://ganglia.sourceforge.net/>

and measure the average value as well as variation trend over time.

Recovery Latency (RL). We measure *RL* as the duration from the moment when a failure happens to the moment when system completes replaying the last tuple. We define two versions of measurements. As shown in Fig. 8(a), because it is a bit tricky to determine when the rollback begins, we include the time for failure detection and standby deployment into rollback latency. Moreover, replay phase works in either parallel or serial with rollback phase. Therefore, Equation 2 gives a *fine-grained* definition of *RL*. A *coarse-grained* *RL* can be evaluated indirectly by the evolution of throughput. As illustrated in Fig. 8(b), once failure happens, throughput fluctuates sharply compared to that during runtime. Hence, we can measure *RL* roughly by the duration from the moment when a failure happens to the moment when throughput resumes steady.

$$RL_{fine} = \begin{cases} RL_{rollback} + RL_{replay, parallel} \\ \max(RL_{rollback}, RL_{replay, serial}) \end{cases} \quad (2)$$

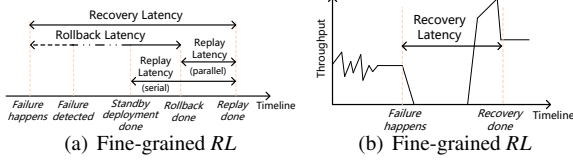


Fig. 8 Two versions of definitions of recovery latency.

4.2 Workloads

Previous stream benchmarks cannot be applied here since they concentrate on the performance evaluation during failure-free runtime. Hence, we design workloads customized for fault tolerance.

4.2.1 Use cases

Covering all the streaming workloads is quite impractical, especially when most available workloads are not published in detail. Hence, we choose and implement three typical stream processing applications, as depicted in Fig. 9. Aggregation is one of the basic operations in real-world scenarios. A widespread use case is streaming *Word Count* (WC). As shown in Fig. 9(a), stateful Counter continuously updates the number of occurrences of each word from stateless Splitter. Numerical computation is also common in stream processing, and the amount of computation varies in different applications, such as CPU-bound model training in machine learn-

ing algorithms. Hence, we create a synthetic *PI Calculation* (PI) as shown in Fig. 9(b). We use Gregory-Leibniz series formula $\frac{\pi}{4} = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$ to control the amount of computation via n . Last we generate linear *Dummy Topology* (DT) with different numbers of operators to control the depth of topology, as shown in Fig. 9(c). Each operator in dummy topologies has no real computing task, but receives and sends tuples.

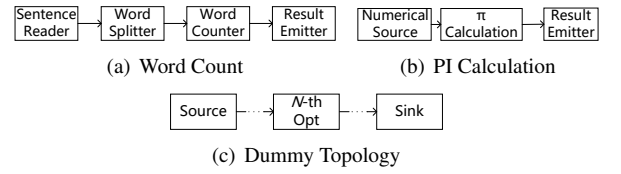


Fig. 9 Use cases of stream applications.

4.2.2 Workload suites

We figure out 6 knobs that have great influence on the quality of fault tolerance as shown in Table 5: 3 for stream application characteristics and 3 for checkpointing configurations. *State* denotes the size in byte of checkpoints. During the experiments, we find that if we enable the window mode of operators, checkpointing can happen at any moment within a time window. As a result, state of each checkpoint is frequently haphazard, baffling the evaluation of its impacts. Therefore, we artificially simulate *State* by a string variable in each stateful operators, and conduct the experiments under full-history mode. *Computing intensity* controls the computation amount of operators and is configured as various execution rounds of PI calculation. *Topology depth* is the number of operators in the longest path of a topology. *Checkpoint interval* specifies the time interval between two consecutive checkpointing requests. *Checkpoint synchronicity* defines whether checkpointing procedure executes synchronously with normal processing. *Barrier alignment* defines whether barriers are aligned.

The knobs, together with use cases mentioned above, compose 6 workload suites listed in Table 6, each of which focuses on distinct effects of knobs on the qualities of fault tolerance. **Suite 1-4** measure the extra runtime overhead of each DSPS incurred by fault tolerance; **Suite 5-6** evaluate the recovery efficiency of each DSPS after randomly killing a worker process.

Table 5 Configurable knobs for workloads

Object	Knob	Abbr	Unit	Value
Application	State size	State	MB	1, 10, 15, 30
	Computation intensity	<i>CPI</i>	round	0(low), 2000(medium), 5000(high)
	Topology depth	<i>TD</i>	# of operators	2, 20
Checkpoint	Checkpoint interval	<i>CI</i>	second	NC(disabled), 1, 30, 45, 60
	Checkpoint synchronicity	<i>CS</i>	-	sync, async
	Barrier alignment	<i>BA</i>	-	true, false

Table 6 Workload suites with distinct purposes

Workload Suite	Use Case	Description	Object
1	<i>WC</i>	Effects of <i>CI</i> on <i>NL</i> and <i>CPU</i> for applications with different <i>State</i>	Runtime Overhead
2	<i>PI</i>	Effects of <i>CS</i> on <i>NL</i> and <i>CPU</i> for applications with different <i>CPI</i>	
3	<i>WC</i>	Effects of <i>BA</i> on <i>NL</i> and <i>CPU</i> when processing data with different <i>Skew</i>	
4	<i>WC</i>	<i>NL</i> under different workload fluctuations	Recovery Efficiency
5	<i>DT</i>	Effects of <i>TD</i> on <i>RL</i>	
6	<i>WC</i>	Effects of <i>CI</i> on <i>RL</i>	

4.3 Datasets

We use three kinds of datasets: a real-world English novel collection crawled from Project Gutenberg⁵⁾, and a synthetic text dataset with different skew distributions of words. Table 7 shows two knobs for datasets. To ensure each system of stable running and avoid backpressure, we keep the data generation at a steady rate 5,000 tuples/s by default⁶⁾ for all the workload suites except for **Suite 4**. And we lower the input rate down to 500 tuples/s on Storm in **Suite 6** due to its performance bottleneck. Skew distribution is common in real world, hence we simulate uniform (*Skew* = 0) and highly-skewed (*Skew* = 1) distributions, respectively.

Table 7 Configurable Knobs for Data Generation

Knob	Abbr	Unit	Value
Input Rate	<i>IR</i>	tuples/s	500, 5000 , [0,7500]
Skew Distribution	<i>Skew</i>	-	0 , 1

⁵⁾ <https://www.gutenberg.org/>

⁶⁾ The setting of input rate is limited to our cluster performance.

4.4 Experimental results

Setup. We perform the experiments on a 5-node cluster where 1 node is equipped with 24 Intel Xeon E5-2620 CPUs and 31GB of RAM, and the others with 8 Intel Xeon E5606 CPUs and 94GB of RAM. The nodes are inter-connected via Gigabit Ethernet and run a CentOS Linux operating system with kernel version 6.5.0. Each set of experiments is executed 5 times to get the stable average results. Since whatever the type of failure is, the final consequence is that some components are unavailable. Hence, we roughly simulate failures by killing worker processes randomly.

4.4.1 Fault tolerance routines

Flink. As shown in Fig. 10(a), three operators *src*, *opt* and *snk* together compose a topology, and are partitioned into different slots. *JobManager* is the controller process, and *TaskManager* is the worker process. *src* receives periodic checkpointing requests from *CheckpointCoordinator*, and then injects barriers into data streams. *opt* with multiple input streams doesn't make checkpoints until it receives barriers with the same ID from all its input streams. When one barrier arrives, *opt* suspends the corresponding input stream and buffers the follow-up tuples into the input queue. Such a step is called *barrier alignment*. It can be turned off, but thereby precise recovery will not be guaranteed. Once all the barriers with the same version ID arrive, *opt* can either block the normal processing to make checkpoints synchronously, or apply the "copy-on-write" technique to make checkpoints asynchronously. When a failure occurs to *TaskManager* (i.e., all the operators it manages fail as well), the recovery procedure is illustrated in Fig. 10(b). Other operators that communicate with the failed ones receive no responses, and then inform *JobManager*. *JobManager* marks the job as *Failing* and broadcasts a *Canceling* message to each alive *TaskManager*. Once received, *TaskManager* forcibly terminates all the operators it manages and sends back a *Canceled* message. *JobManager* will never receive a *Canceled* message from the failed *TaskManager*, and then marks the job as *Restarting* when the threshold of waiting time is reached. The whole topology will be reset and restore *Running* status. Accordingly, all the operators redeployed retrieve their own last checkpoints to rollback the state. Once *src* finishes the rollback, it begins to fetch the tuples that were processed but not reflected into the latest checkpoint.

Storm. We use the same topology to discuss the fault tolerance mechanism of Storm, which is inspired by that of Flink but has some differences. As Fig. 10(c) shows, *Supervi-*

We perform *WC* on the real-world novel dataset to investigate the effects of *CI* on *NL* and *CPU* for tasks with different *State*. Since Storm cannot function properly with highly frequent checkpointing and large state, we don't run it with 1s-*CI* and 30MB-*State*. As shown in Fig. 11, for Flink, more frequent checkpointing means more interference on normal processing and more thread scheduling. When checkpointing executes synchronously with normal processing, larger *State* incurs longer pause of normal processing to make and transmit checkpoints. Hence, both *NL* and *CPU* are proportional to *State*, but inversely proportional to *CI*. For Storm, under a certain *CI*, *NL* and *CPU* have a linear relational with *State*.

⁷⁾ <https://redis.io/>

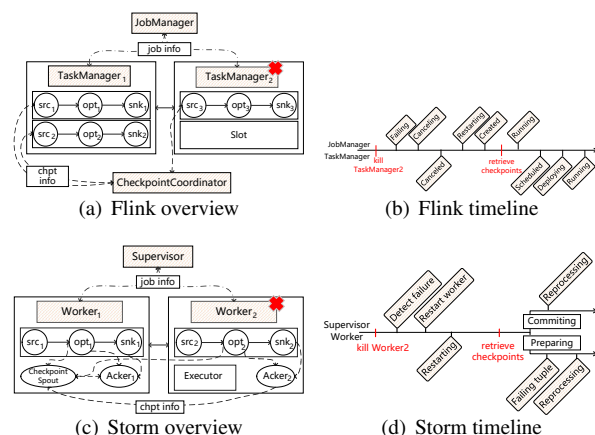


Fig. 10 Fault tolerance routine of Flink and Storm.

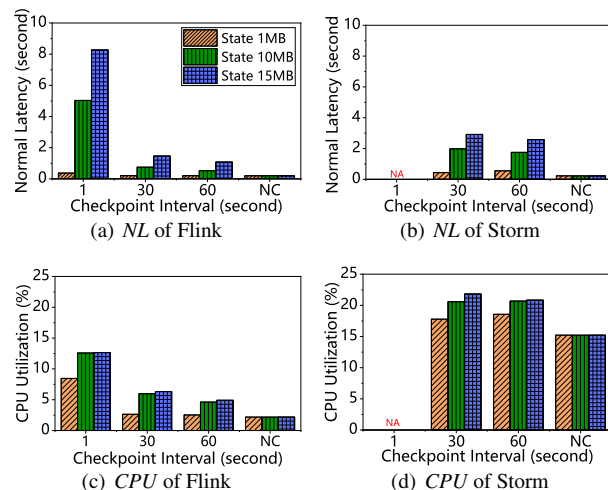


Fig. 11 The effects of *State* and *CI* on *NL* and *CPU*.

4.4.3 Workload suite 2

We perform *PI* on the synthetic dataset with uniform distribution to investigate the effects of *CS* on *NL* and *CPU* for applications with different *CPI*. Since checkpointing by default executes synchronously with normal processing, and that checkpointing messages are not aligned in Storm, we only evaluate Flink here. As shown in Fig. 12(a)-12(c), if checkpointing executes asynchronously, new incoming tuples will be processed in time; otherwise, they will be buffered in input queues of each operator until checkpointing is finished. Hence, asynchronous checkpointing can optimize the increase of *NL*.

Moreover, we evaluate *CPU* under different *CPI*. As the total computation amount is constant, the average *CPU* of asynchronous checkpointing is approximate to that of synchronous checkpointing, as shown in Fig. 12(d)-12(f). For

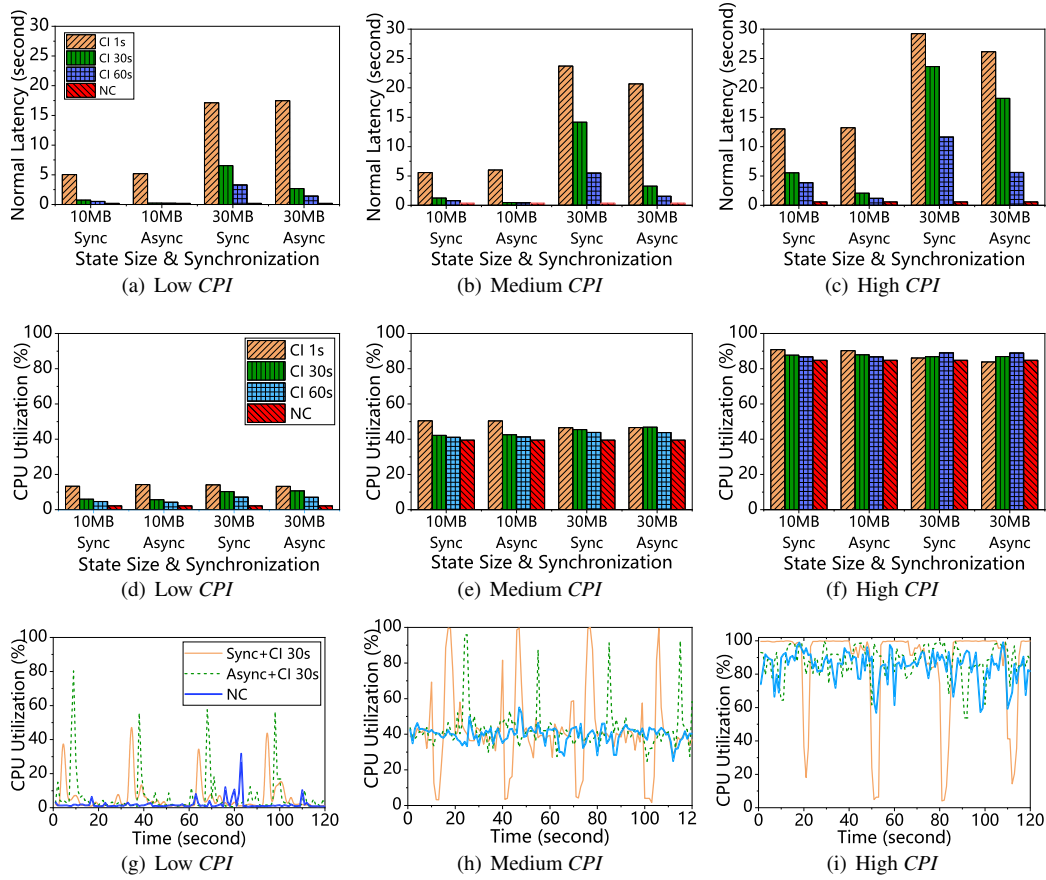


Fig. 12 The effects of CS and CPI on NL and CPU.

further analysis, we evaluate *CPU* trend over time. As illustrated in Fig. 12(g)-12(i), with the increase of *CPI*, *CPU* of synchronous checkpointing changes more violently. Once checkpointing is trigger, each operator suspends its normal processing, which leads to an obvious trough of *CPU*. When checkpointing is finished, the buffered new incoming tuples are first processed, which leads to a peak. In Fig. 12(i), there is no peak but a longer continuously-high period in that *CPU* is already extremely high due to heavy *CPI*.

We conclude that asynchronous checkpointing gains lower runtime overhead, but this predominance is weakened under heavy computation amount and highly-frequent checkpointing.

4.4.4 Workload suite 3

We conduct *WC* on the synthetic dataset to evaluate the effects of *BA* on *NL* and *CPU* when processing input streams with different *Skew*. As shown in Fig. 13, with uniform distribution, effects on *NL* have little difference whether barriers are aligned. However, when input streams are highly-

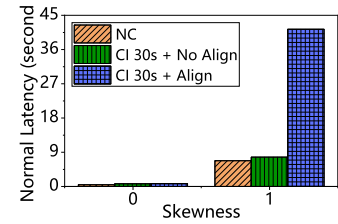


Fig. 13 The effects of *Skew* and *BA* on *NL* in Flink.

skewed, arrival time of barriers on different streams differs sharply with each other, which leads to longer blocking of the stream on which the barrier arrives first, and accordingly higher *NL*. Fig. 14 demonstrates *CPU* trend over time under different *Skew*. We also compare high-load nodes with low-load nodes. As Fig. 14(a)-14(b) demonstrate, when *Skew* is low, there is little difference on *CPU* whether barriers are aligned or not. But with severe skewness, temporary blocking of some stream causes massive tuples buffered, which leads to an obvious period of peak usage of *CPU*, as shown in Fig. 14(c)-14(d).

We conclude that aligning barriers incurs higher normal

latency and unstable CPU status when input streams are of highly-skewed distribution.

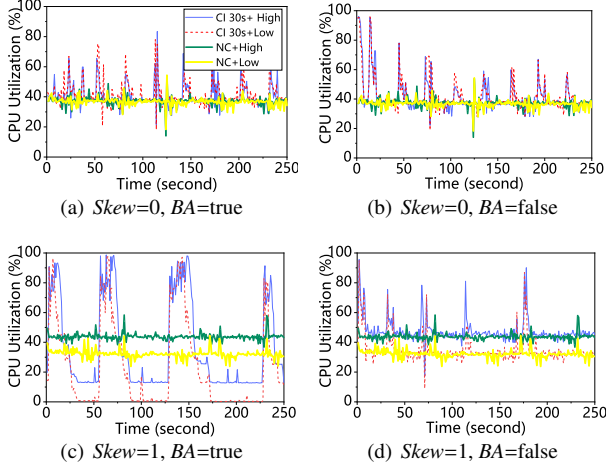


Fig. 14 The effects of Skew and BA on NL and CPU in Flink.

4.4.5 Workload suite 4

We conduct WC on the real-world novel dataset with dynamic input rates to evaluate the NL of Flink. Storm displays the similar performance variation but with much worse metrics compared to Flink. In order to show the curves clearly, we just demonstrate the results of Flink. As shown in Fig. 15, we perform the experiments under two kinds of workload fluctuations. Fig. 15(a) shows NL over time under low fluctuation where only one workload peak happens around the 2nd checkpointing. Fig. 15(b) shows NL under high fluctuation where we have several workload peaks. We find that checkpointing mechanisms with static CI exacerbate the performance degradation (i.e., continuously high-latency status after the 2nd checkpointing) if workload fluctuates sharply and frequently. Hence, we conclude that static CI is not efficient enough to satisfy the real-world dynamic workloads.

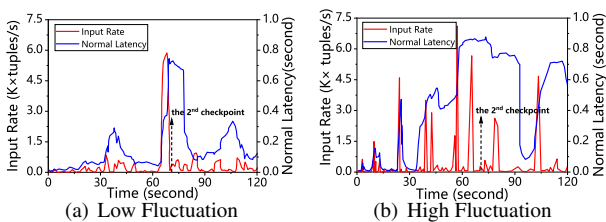


Fig. 15 NL in Flink under dynamic input streams: CI=30s, State=1MB.

4.4.6 Workload suite 5

We conduct DT on synthetic dataset with uniform distribution to evaluate the effects of TD on rollback latency in Flink. During the experiments, we find that Storm has highly similar performance to Flink when we adjust the topology depth to investigate its effects on the rollback latency. Therefore, we omit the presentation of evaluation results of Storm. After system warm-up, we simulate failures by randomly killing a worker process just before the next checkpointing request is issued. As discussed in Section 1, *JobManager* decides the failure of a failed *TaskManager*, if it receives no responses from *TaskManager* when the threshold of waiting time is reached. As shown in Fig. 16, longer topology lightly increases the rollback latency, but is bound to the timeout threshold of failure detection.

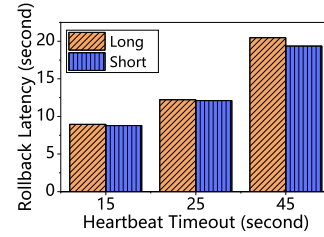


Fig. 16 The effects of TD on NL in Flink.

4.4.7 Workload suite 6

We perform WC on synthetic dataset with uniform distribution to evaluate the effects of CI on recovery latency for applications with different State. Input rate for Storm is adjusted to 500 tuples/s because its mediocre performance. Fig. 17 presents the fine-version recovery latency of Flink and Storm. We observe that the rollback latency of Flink is always bound to 45 seconds, having nothing to do with State and CI. After reviewing system logs, we find that the default time-out threshold of failure detection is 45 seconds. However, as Fig. 17(b) and 17(d) show, the replay latency of Flink has a linear relationship with CI, but is not affected by State; while State has greater influence on the volume of replayed tuples than CI. We analyze logs and observe that the replay phase begins before all the operators have finished the rollback phase. Once the source operator has finished its rollback, it begins to fetch tuples from DB. The bigger State implies more time downstream operators spend on rollback. New incoming tuples are buffered before downstream operators have finished the rollback, which may cause back-pressure and consequently lead to higher replay latency. For

Storm, RL is proportional to CI as shown in Fig. 17(d), but almost not affected by $State$. We figure out that RL is mainly affected by batch-acking and tuple replaying. When CI is relatively large, the impact of batch-acking outweighs that of tuple replaying. Hence, when failure happens in preparing phase, RL is higher than that at committing phase. While for the smaller CI , we draw an opposite conclusion.

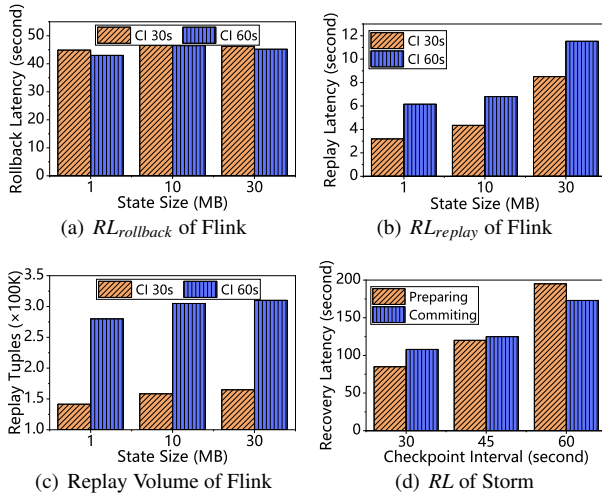


Fig. 17 The effects of $State$ and CI on fine-version RL of Flink and Storm.

Fig. 18 shows the coarse-version recovery latency of Flink and Storm. The first dotted arrow marks the moment when failure occurs, and the second one marks the moment when the system restores the steady throughput. For Flink, after failure is detected, source operators of Flink stop fetching tuples from data broker, which results in a temporary zero throughput. After source operators finish the rollback phase, tuples that were emitted from the latest checkpoint to the failure point are re-fetched, leading to wave crests. For Storm, source operators keep emitting tuples downstream even when failure happens. The wave crests imply the replaying of tuples which are actively failed by downstream operators.

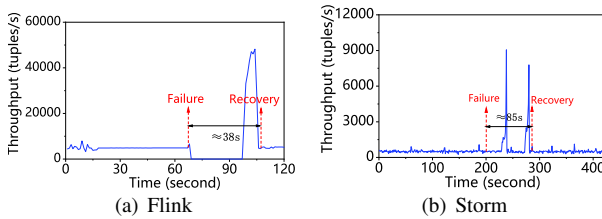


Fig. 18 The coarse-version RL of Flink and Storm.

We conclude that (1) Flink recovers much more efficiently than Storm. Even all the operators are forced to be redeployed in Flink, it has far less recovery latency than Storm;

(2) Checkpoint interval has great effects on recovery latency, as larger interval indicates more tuples to be replayed.

5 Conclusions and future research directions

Stream processing has been with large-scale adoption in industry and research community, owing to the real-time reaction and quick adaption to data changes. In this work, we discuss and classify fault tolerance approaches in stream processing systems, which are especially urgent in distributed or cloud-based environment. For each type of approach, we concretely summarize the vital design principles, list the advantages/disadvantages, and describe the application in the representative system. For active fault tolerance, we are the first to develop a taxonomy in terms of replica coordination. Due to the complicated coordination and multiplied resource consumption, it isn't practical to be applied in production deployment alone, but could be collaborated with passive fault tolerance in an efficient way. For passive fault tolerance approaches, we dissect plentiful checkpointing protocols and also develop a taxonomy in terms of four key aspects of checkpointing. For its conservative resource consumption, such approaches are more preferable for modern stream processing systems and applications though they may tolerate higher recovery latency. Moreover, we develop an evaluation framework to evaluate the quality of fault tolerance, and we find some nontrivial conclusions which have not been exposed by any previous work: (1) in Storm, batch-acking mechanism counteracts the optimization of incremental checkpointing; (2) in Flink, the preponderance of asynchronous checkpointing fades away when the processing task is CPU-bound. For future research directions in developing fault-tolerant SPSs, we summarize three recent trends as follows.

Adaptive Checkpointing. As we find, most of the current passive fault tolerance approaches are developed with periodical checkpointing. In reality, streaming data has the characteristics of dynamics, and its source and incoming rate are hard to predicate. Workload variance is also a common phenomenon. Periodical checkpointing is easy to implement, but the fixed checkpoint interval incurs inflexibility. Checkpointing at the point of data burst will severely degrade the system performance. Further, the choice of checkpoint interval is also a tricky problem for different applications with different requirements. As a future work, we encourage the investigation about adaptive checkpointing protocols which choose the right time for checkpoints.

Integration of Modern Hardware. We see an emerging field of modern hardware architectures and devices, such as RDMA (remote direct memory access), NVRAM (non-volatile RAM storage), which demonstrate new properties and present new opportunities for stream processing systems to further promote the performance. For instance, NVRAM can combine the advantages of DRAM and disk, namely the characteristics of fast sequential access and persistence. However, leveraging modern hardware in SP systems reveals challenges as well, for example, I/O operations can be more complicated. Constantin et al. [77] propose a hardware-oriented cost model for query optimization of stream processing. Steffen et al. [78] conduct extensive experiments to explore the efficient stream processing on modern hardware. Therefore, exploiting fault tolerance techniques on modern hardware is also an important problem worthy of study.

Parallel Recovery. Modern stream processing systems based on continuous-operator model usually do not support dynamic adjustments of operator parallelism and data partition during runtime. Even resetting the entire streaming job after failures, the query execution plan remains the same, and operators are recovered serially. Therefore, to enable dynamic scale-out and scale-in of SP systems is essential provides the possibility of parallel recovery.

Acknowledgments The work is supported by the National Key Research and Development Plan Project (No. 2018YFB1003404).

References

1. F. Naughton Jeffrey, J. DeWitt David, Maier David, Aboulmaga Ashraf, Chen Jianjun, Galanis Leonidas, Kang Jaewoo, Krishnamurthy Rajasekar, Luo Qiong, Prakash Naveen, Ramamurthy Ravishankar, Shanmugasundaram Jayavel, Tian Feng, Tuft Kristin, Viglas Stratis, Wang Yuan, Zhang Chun, Jackson Bruce, Gupta Anurag, Kumar, and Chen Rushan. The niagara internet query system. *Data Eng. Bull.*, 24(2):27–33, 2001.
2. J. Abadi Daniel, Carney Donald, Çetintemel Ugur, Cherniack Mitch, Convey Christian, Lee Sangdon, Stonebraker Michael, Tatbul Nesime, and B. Zdonik Stanley. Aurora: A new model and architecture for data stream management. *VLDB J.*, 12(2):120–139, 2003.
3. Rajeev Motwani, Jennifer Widom, Arvind Arasu, Brian Babcock, Shivnath Babu, Mayur Datar, Gurmeet Singh Manku, Chris Olston, Justin Rosenstein, and Rohit Varma. Query processing, approximation, and resource management in a data stream management system. In *Proceedings of the 1st International Conference on Innovative Data Systems Research*, 2003.
4. Chandrasekaran Sirish, Cooper Owen, Deshpande Amol, J. Franklin Michael, M. Hellerstein Joseph, Hong Wei, Krishnamurthy Sailesh, Madden Samuel, Raman Vijayshankar, Reiss Frederick, and A. Shah Mehul. Telegraphicq: Continuous dataflow processing for an uncertain world. In *Proceedings of the 1st Biennial Conference on Innovative Data Systems Research*, 2003.
5. Thomas Heinze, Leonardo Aniello, Leonardo Querzoni, and Zbigniew Jerzak. Cloud-based data stream processing. In *Proceedings of the 8th ACM International Conference on Distributed Event-Based*, pages 238–245, 2014.
6. Mitch Cherniack, Hari Balakrishnan, Magdalena Balazinska, Donald Carney, Ugur Çetintemel, Ying Xing, and Stanley B. Zdonik. Scalable distributed stream processing. In *Proceedings of the 1st International Conference on Innovative Data Systems Research*, 2003.
7. J. Abadi Daniel, Ahmad Yanif, Balazinska Magdalena, Çetintemel Ugur, Cherniack Mitch, Hwang Jeong-Hyon, Lindner Wolfgang, Maskey Anurag, Rasin Alex, Ryvkina Esther, Tatbul Nesime, Xing Ying, and B. Zdonik Stanley. The design of the borealis stream processing engine. In *Proceedings of the 2nd Biennial Conference on Innovative Data Systems Research*, pages 277–289, 2005.
8. Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th USENIX Symposium on Operating System Design and Implementation*, pages 137–150, 2004.
9. Toshniwal Ankit, Taneja Siddharth, Shukla Amit, Ramasamy Karthikeyan, M. Patel Jignesh, Kulkarni Sanjeev, Jackson Jason, Gade Krishna, Fu Maosong, Donham Jake, Bhagat Nikunj, Mittal Sailesh, and V. Ryaboy Dmitriy. Storm@twitter. In *Proceedings of the ACM International Conference on Management of Data*, pages 147–156, 2014.
10. Zaharia Matei, Das Tathagata, Li Haoyuan, Hunter Timothy, Shenker Scott, and Stoica Ion. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, pages 423–438, 2013.
11. Carbone Paris, Katsifodimos Asterios, Ewen Stephan, Markl Volker, Haridi Seif, and Tzoumas Kostas. Apache flink™: Stream and batch processing in a single engine. *Data Eng. Bull.*, 38(4):28–38, 2015.
12. A. Noghabi Shadi, Paramasivam Kartik, Yi Pan, Ramesh Navina, Bringham Jon, Gupta Indranil, and H. Campbell Roy. Stateful scalable stream processing at linkedin. *Proc. VLDB Endow.*, 10(12):1634–1645, 2017.
13. Google cloud. <http://cloud.google.com/>.
14. Amazon ec2. <https://aws.amazon.com/ec2/>.
15. Microsoft azure. <https://azure.microsoft.com/>.
16. Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. Understanding network failures in data centers: measurement, analysis, and implications. In *Proceedings of the ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*, pages 350–361, 2011.
17. Dean Jeff. Handling large datasets at google: Current systems and future directions. In *Data-Intensive Computing Symposium*, 2008.
18. Vishwanath Kashi, Venkatesh and Nagappan Nachiappan. Characterizing cloud computing hardware reliability. In *Proceedings of the 1st Symposium on Cloud Computing*, pages 193–204, 2010.
19. Shankland Stephen. Google spotlights data center inner workings. <https://www.cnet.com/news/>

- google-spotlights-data-center-inner-workings/.
20. In pictures: The worst cloud outages of 2013. <https://www.pcworld.idg.com.au/slideshow/466392/pictures-worst-cloud-outages-2013-far/>.
 21. A. Shah Mehul, M. Hellerstein Joseph, and A. Brewer Eric. Highly-available, fault-tolerant, parallel dataflows. In *Proceedings of the 30th ACM International Conference on Management of Data*, pages 827–838, 2004.
 22. Balazinska Magdalena, Balakrishnan Hari, Madden Samuel, and Stonebraker Michael. Fault-tolerance in the borealis distributed stream processing system. In *Proceedings of the 31st ACM International Conference on Management of Data*, pages 13–24, 2005.
 23. Hwang Jeong-Hyon, Çetintemel Ugur, and B. Zdonik Stanley. Fast and highly-available stream processing over wide area networks. In *Proceedings of the 24th IEEE International Conference on Data Engineering*, pages 804–813, 2008.
 24. Martin André, Fetzer Christof, and Brito Andrey. Active replication at (almost) no cost. In *Proceedings of the 30th Symposium on Reliable Distributed Systems*, pages 21–30, 2011.
 25. Bellavista Paolo, Corradi Antonio, Kotoulas Spyros, and Reale Andrea. Adaptive fault-tolerance for dynamic resource provisioning in distributed stream processing systems. In *Proceedings of the 17th Springer International Conference on Extending Database Technology*, pages 85–96, 2014.
 26. Hwang Jeong-Hyon, Xing Ying, Çetintemel Ugur, and B. Zdonik Stanley. A cooperative, self-configuring high-availability solution for stream processing. In *Proceedings of the 23rd IEEE International Conference on Data Engineering*, pages 176–185, 2007.
 27. Kwon YongChul, Balazinska Magdalena, and G. Greenberg Albert. Fault-tolerant stream processing using a distributed, replicated file system. *PVLDB*, 1(1):574–585, 2008.
 28. Gu Yu, Zhang Zhe, Ye Fan, Yang Hao, Kim Minkyong, Lei Hui, and Liu Zhen. An empirical study of high availability in stream processing systems. In *Proceedings of the 9th International Middleware Conference*, page 23, 2009.
 29. Sebeopou Zoe and Magoutis Kostas. Cec: Continuous eventual checkpointing for data stream processing operators. In *Proceedings of International Conference on Dependable Systems and Networks*, pages 145–156, 2011.
 30. Fernandez Raul, Castro, Migliavacca Matteo, Kalyvianaki Evangelia, and R. Pietzuch Peter. Integrating scale out and fault tolerance in stream processing using operator state management. In *Proceedings of the ACM International Conference on Management of Data*, pages 725–736, 2013.
 31. Koldehofe Boris, Mayer Ruben, Ramachandran Umakishore, Rothermel Kurt, and Völz Marco. Rollback-recovery without checkpoints in distributed event processing systems. In *Proceedings of the 7th ACM International Conference on Distributed Event-Based Systems*, pages 27–38, 2013.
 32. Qun Huang and Patrick P. C. Lee. Toward high-performance distributed stream processing via approximate fault tolerance. *PVLDB*, 10(3):73–84, 2016.
 33. Hongliang Li, Jie Wu, Zhen Jiang, Xiang Li, and Xiaohui Wei. Minimum backups for stream processing with recovery latency guarantees. *IEEE Transactions on Reliability*, 66(3):783–794, 2017.
 34. Wei Xiaohui, Liu Zhiliang, Zhuang Yuan, Li Hongliang, and Li Xiang. Adaptive checkpoint mechanism supporting large-scale stream data processing. *Journal of Jilin University (Engineering and Technology Edition)*, 47(1):199–207, 2017.
 35. Zhang Zhe, Gu Yu, Ye Fan, Yang Hao, Kim Minkyong, Lei Hui, and Liu Zhen. A hybrid approach to high availability in stream processing systems. In *Proceedings of the 30th IEEE International Conference on Distributed Computing Systems*, pages 138–148, 2010.
 36. Heinze Thomas, Zia Mariam, Krahn Robert, Jerzak Zbigniew, and Fetzer Christof. An adaptive replication scheme for elastic data stream processing systems. In *Proceedings of the 9th International Conference on Distributed Event-Based Systems*, pages 150–161, 2015.
 37. Su Li and Zhou Yongluan. Tolerating correlated failures in massively parallel stream processing engines. In *Proceedings of the 32nd IEEE International Conference on Data Engineering*, pages 517–528, 2016.
 38. Li Su and Yongluan Zhou. Passive and partially active fault tolerance for massively parallel stream processing engines. *IEEE Trans. Knowl. Data Eng.*, 31(1):32–45, 2019.
 39. Martin André, Smanec Tiaraju, Dietze Tobias, Brito Andrey, and Fetzer Christof. User-constraint and self-adaptive fault tolerance for event stream processing systems. In *Proceedings of the 45th IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 462–473, 2015.
 40. B. Schneider Fred. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 22(4):299–319, 1990.
 41. Chandra and Robert Griesemer Tushar, Deepak and Redstone Joshua. Paxos made live: An engineering perspective. In *Proceedings of the 26th ACM Symposium on Principles of Distributed Computing*, pages 398–407, 2007.
 42. E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.*, 34(3):375–408, 2002.
 43. Balazinska Magdalena, Hwang Jeong-Hyon, and A. Shah Mehul. Fault-tolerance and high availability in data stream management systems. In *Encyclopedia of Database Systems*, pages 1109–1115, 2009.
 44. Sampaio Gradwohl Andre, Leon, Senger Hermes, Arantes Luciana, and Sens Pierre. Comparing distributed online stream processing systems considering fault tolerance issues. *Journal of Emerging Technologies in Web Intelligence*, 6(2):174–179, 2014.
 45. Anis Uddin Nasir, Muhammad. Fault tolerance for stream processing engines. *arXiv preprint*, abs/1605.00928, 2016.
 46. Arasu Arvind, Cherniack Mitch, F. Galvez Eduardo, Maier David, Maskey Anurag, Ryvkina Esther, Stonebraker Michael, and Tibbetts Richard. Linear road: A stream data management benchmark. In *Proceedings of the 30th ACM International Conference on Very Large Data Bases*, pages 480–491, 2004.
 47. Lu Ruirui, Wu Gang, Xie Bin, and Hu Jingtong. Streambench: Towards benchmarking modern distributed stream computing frame-

- works. In *Proceedings of the 7th IEEE/ACM International Conference on Utility and Cloud Computing*, pages 69–78, 2014.
48. Sanket Chintapalli, Derek Dagit, Bobby Evans, Reza Farivar, Thomas Graves, Mark Holderbaugh, Zhuo Liu, Kyle Nusbaum, Kishorkumar Patil, Boyang Peng, and Paul Poulosky. Benchmarking streaming computation engines: Storm, flink and spark streaming. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops*, pages 1789–1792, 2016.
 49. Jamie Grier. Extending the yahoo! streaming benchmark. <https://www.ververica.com/blog/extending-the-yahoo-streaming-benchmark>, 2016.
 50. Yangjun Wang. Stream processing systems benchmark: Streambench. Master’s thesis, Aalto University, 2016.
 51. Shukla Anshu, Chaturvedi Shilpa, and Simmhan Yogesh. Riotbench: A real-time iot benchmark for distributed stream processing platforms. *arXiv preprint*, abs/1701.08530, 2017.
 52. Bordin Maycon, Viana. *A Benchmark Suite for Distributed Stream Processing Systems*. PhD thesis, Universidade Federal do Rio Grande Do Su, 2017.
 53. Jeyhun Karimov, Tilmann Rabl, Asterios Katsifodimos, Roman Samarev, Henri Heiskanen, and Volker Markl. Benchmarking distributed stream data processing systems. In *Proceedings of the 34th IEEE International Conference on Data Engineering*, pages 1507–1518, 2018.
 54. Venkataraman Shivaram, Panda Aurojit, Ousterhout Kay, Armbrust Michael, Ghodsi Ali, J. Franklin Michael, Recht Benjamin, and Stolica Ion. Drizzle: Fast and adaptable stream processing at scale. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles*, pages 374–389, 2017.
 55. Akidau Tyler, Bradshaw Robert, Chambers Craig, Chernyak Slava, Fernández-Moctezuma Rafael, Lax Reuven, McVeety Sam, Mills Daniel, Perry Frances, Schmidt Eric, and Whittle Sam. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proc. VLDB Endow.*, 8(12):1792–1803, 2015.
 56. Aol search data. <http://www.researchpipeline.com/mediawiki/>.
 57. The caida ucsd statistical information for the caida anonymized internet traces. http://www.caida.org/data/passive/passive_trace_statistics.xml.
 58. Sense your city: Data art challenge, 2015.
 59. The research perspective ltd. cer smart metering project., 2012.
 60. Brian Donovan and Daniel B. Work. Using coarse GPS data to quantify city-scale transportation system resilience to extreme events. *CoRR*, abs/1507.06011, 2015.
 61. Project gutenber. <http://www.gutenberg.org/>.
 62. 1998 worldcup. <https://ita.ee.lbl.gov/html/contrib/WorldCup.html>.
 63. Beijing taxi traces. <http://anrg.usc.edu/www/downloads/>.
 64. Hwang Jeong-Hyon, Balazinska Magdalena, Rasin Alex, Çetintemel Ugur, Stonebraker Michael, and B. Zdonik Stanley. High-availability algorithms for distributed stream processing. In *Proceedings of the 21st IEEE International Conference on Data Engineering*, pages 779–790, 2005.
 65. Peter A. Tucker and David Maier. Exploiting punctuation semantics in data streams. In *Proceedings of the 18th IEEE International Conference on Data Engineering*, page 279, 2002.
 66. Chandy K. Mani and Lamport Leslie. Distributed snapshots: Determining global states of distributed systems. *ACM Trans. Comput. Syst.*, 3(1):63–75, 1985.
 67. Wang Yi-Min, Chung Pi-Yu, Lin In-Jen, and Fuchs W. Kent. Checkpoint space reclamation for uncoordinated checkpointing in message-passing systems. *Trans. Parallel Distrib. Syst.*, 6(5):546–554, 1995.
 68. Neumeyer Leonardo, Robbins Bruce, Nair Anish, and Kesari Anand. S4: Distributed stream computing platform. In *Proceedings of the 10th IEEE International Conference on Data Mining Workshops*, pages 170–177, 2010.
 69. Brian Randell. System structure for software fault tolerance. *IEEE Trans. Software Eng.*, 1(2):221–232, 1975.
 70. Gordon Murray Derek, McSherry Frank, Isaacs Rebecca, Isard Michael, Barham Paul, and Abadi Martín. Naiad: A timely dataflow system. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, pages 439–455, 2013.
 71. Akidau Tyler, Balikov Alex, Bekiroglu Kaya, Chernyak Slava, Haberman Josh, Lax Reuven, McVeety Sam, Mills Daniel, Nordstrom Paul, and Whittle Sam. Millwheel: Fault-tolerant stream processing at internet scale. *Proc. VLDB Endow.*, 6(11):1033–1044, 2013.
 72. Qian Zhengping, He Yong, Su Chunzhi, Wu Zhuojie, Zhu Hongyu, Zhang Taizhi, Zhou Lidong, Yu Yuan, and Zhang Zheng. Timestream: Reliable stream computation in the cloud. In *Proceedings of the 8th Eurosys Conference*, pages 1–14, 2013.
 73. Bharat K. Bhargava and Shy-Renn Lian. Independent checkpointing and concurrent rollback for recovery in distributed systems - an optimistic approach. In *Proceedings of the 7th Symposium on Reliable Distributed Systems*, pages 3–12, 1988.
 74. I. Feldman Stuart and B. Brown Channing. Igor: A system for program debugging via reversible execution. In *Proceedings of the ACM SIGPLAN and SIGOPS Workshop on Parallel and Distributed Debugging*, pages 112–123, 1988.
 75. Ghemawat Sanjay, Gobioff Howard, and Leung Shun-Tak. The google file system. In *Proceedings of the 19th ACM SOSP Symposium on Operating Systems Principles*, pages 29–43. ACM, 2003.
 76. Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *Proceedings of the 26th IEEE Symposium on Mass Storage Systems and Technologies*, pages 1–10, 2010.
 77. Constantin Pohl, Philipp Götze, and Kai-Uwe Sattler. A cost model for data stream processing on modern hardware. In *Proceedings of the International Workshop on Accelerating Analytics and Data Management Systems Using Modern Processor and Storage Architectures*, 2017.
 78. Steffen Zeuch, Sebastian Breß, Tilmann Rabl, Bonaventura Del Monte, Jeyhun Karimov, Clemens Lutz, Manuel Renz, Jonas Traub, and

Volker Markl. Analyzing efficient stream processing on modern hardware. *Proc. VLDB Endow.*, 12(5):516–530, 2019.



Xiaotong Wang is a doctoral student of East China Normal University, under the supervision of Prof. Rong Zhang. She received her BE degree in computer science from Nanjing Normal University, China in 2012. Her current research interests include data management and distributed stream

processing.



Chunxi Zhang is a doctoral student of East China Normal University, under the supervision of Prof. Rong Zhang. She received her BE degree in software engineering from Shandong University of Science and Technology, China in 2011. Her current research interest is performance evaluation of databases,

including intensive workload generation, contention simulation and isolation validation.



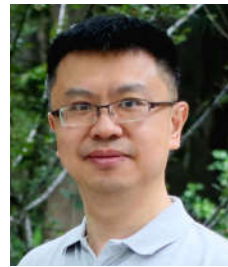
Junhua Fang received his PhD degree from East China Normal University, under the supervision of Prof. Aoying Zhou. He is an associate professor in Advanced Data Analytics Group at the School of Computer Science and Technology, Soochow University, Suzhou. His research focuses on distributed database and parallel stream processing. He is a member

of China Computer Federation.



Rong Zhang received her PhD degree in computer science from Fudan University, China in 2007. She joined East

China Normal University since 2011 and is currently a professor with the university. From 2007 to 2010, she worked as an expert researcher with NICT, Japan. Her current research interests include knowledge management, distributed data management, and database benchmarking. She is a member of China Computer Federation.



Weining Qian received his PhD degree in computer science from Fudan University, China in 2004. He is a professor and dean of the School of Data Science and Engineering, East China Normal University, China. He is now serving as a standing committee member of Database Technology Committee of

China Computer Federation, and committee member of ACM SIGMOD China Chapter. His research interests include scalable transaction processing, benchmarking big data systems, and management and analysis of massive datasets.



Aoying Zhou is a professor on computer science with East China Normal University (ECNU), where he is heading the School of Data Science and Engineering. Before joining ECNU in 2008, he worked for Fudan University at the Computer Science Department for 15 years. He is the winner of the

National Science Fund for Distinguished Young Scholars supported by NSFC and the professorship appointment under Changjiang Scholars Program of Ministry of Education. He is now acting as a vice-director of ACM SIGMOD China and Database Technology Committee of China Computer Federation. He is serving as a member of the editorial boards the VLDB Journal, the WWW Journal, and etc. His research interests include data management, in-memory cluster computing, big data benchmarking, and performance optimization.